

1



An Enterprise IT Renovation Roadmap

This book makes a big promise: It is offering an *IT renovation roadmap*, which will leverage the concepts of Service-Oriented Architectures (SOA) on both the technical and organizational levels in order to create sustainable improvements in IT efficiency and agility. The aim of this roadmap is to strike a good balance between immediate gains on one hand and long-lasting improvements to the enterprise IT landscape on the other. An SOA should increase the capability of an enterprise to address new business requirements on the short term by reusing existing business logic and data models, thus incurring only minimal cost, resource, and time overheads, while minimizing risks, especially when compared to rewriting entire application systems. In addition, an SOA should provide enduring benefits in terms of agility because it provides a long-term strategy for the increase of the flexibility of an IT infrastructure.

This chapter closely looks at the problems faced by enterprise software today, the resulting requirements for an enterprise IT architecture such as an SOA, and how such an architecture can be established on the organizational level.

1.1 Agony Versus Agility

In 2003, Nicolas G. Carr published the heatedly debated article “*IT doesn’t matter*” in the *Harvard Business Review*, claiming that “... like electrical grids or railroads, IT would become a ubiquitous commodity.” Regardless of your position on this issue—whether or not you consider enterprise IT a commodity—enterprises heavily depend on the IT backbone, which is responsible for running almost all processes of modern enterprises, be they related to manufacturing, distribution, sales, customer management, accounting, or any other type of business process. Because of today’s highly competitive global economy, these business processes underlie constant change: Enterprises must constantly sense changes in

market conditions and swiftly adapt their strategies to reflect these changes. Therefore, it is a key requirement for modern enterprise IT that changes in company strategy be reflected quickly and efficiently in the company's IT systems, which are the backbone for executing the strategy.

This is exactly where the enterprise software dilemma starts: Today's enterprise software development almost always suffers from *lack of agility* and from *inefficiency*. This means that enterprises are not able to match business requirements onto underlying IT infrastructure fast enough, effectively limiting the capability of the enterprise to react appropriately to market demands. In addition, the inefficiency of enterprise software development means that the development that is actually done costs too much when compared to the actual output.

If we look at a typical enterprise software system, we can normally observe an initial phase of high productivity and agility, as shown in Figure 1-1. During this Green field phase, the system is built with much new functionality, and initial change requests can be implemented relatively quickly and efficiently. However, after the initial system implementation has been put in place and the first couple of change requests have been executed, the ability to make more changes to the system deteriorates dramatically, and maintenance over time becomes harder and harder.

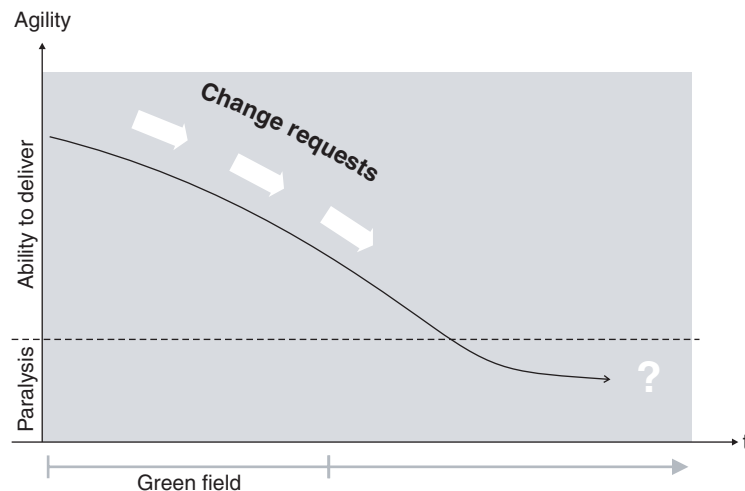


FIGURE 1-1 Change requests reduce the agility of a system over time.

This stagnation phase, which almost any enterprise software system experiences over time, cannot be explained by a single reason—a number of factors contribute to this phenomenon. Some of these reasons are related to software technology, such as the difficulty of making structural changes to an existing code

base. However, most of the reasons are not of a technical nature but rather are related to reasons on the organizational level. For example, after the initial launch phase of a system, the project management and key domain experts are likely to move on to the next project, often leaving the maintenance of the system to less skilled maintenance workers, many times even without doing a proper hand-over. In addition, after the initial phase of euphoria about the new system, it might lose its internal lobby over time and thus the necessary political support within the organization. Tight project budgets often mean that fixes and changes cannot be done properly and are only done on an ad-hoc basis, without taking the existing order of the system into consideration. Typically, there is no time and budget for doing a proper refactoring of the system to ensure its long-term maintainability. Finally, critical structural decisions are often made on the side, without executing proper control—for example, an engineer might quickly write a small batch program to synchronize data between two systems, and 10 years later, a small army of developers is needed to deal with the consequences of this ad-hoc decision.

When looking at enterprise software, we are usually not looking at isolated systems, but at large numbers of systems with complex cross-dependencies that have grown over many years and with a high level of heterogeneity and redundancies. We rarely have sufficient in-house knowledge about the different systems, and we often need to cope with very different design and programming styles. Finally, people are getting used to this situation and are starting to think in terms of workarounds, not in terms of the “right” structures.

Some organizations might have found better ways of coping with these problems than others, but it’s hard to find any organization today that can claim that it has completely sorted out these issues. For this reason, many organizations require an *enterprise IT renovation roadmap* to help providing a sustainable transformation into a more agile IT organization that is able to quickly and efficiently adapt to changing business requirements. This chapter lays out the cornerstones of this roadmap as it is presented in this book.

1.2 Enterprise Software Is a Different Animal

In order to better understand the problems of enterprise software, we need to look at the specific characteristics of it, which are different from those of other types of software, such as system software, desktop applications, embedded systems, scientific software, or video games.

As the name indicates, *enterprise software is tightly coupled with the internal organization, processes, and business model of the enterprise*. Enterprise software underlies both cross-departmental dependencies and external business relationships. Consequently, an architecture for enterprise software must deal with large numbers of different requirements. Many of these requirements are conflicting, while others are unclear. In almost every case, the requirements are a

moving target due to the permanent change of markets, the organization of the enterprise, and its business objectives. It is this involvement in all aspects of the enterprise and the business that makes enterprise software highly complex.

Enterprise applications rarely contain a large amount of complicated algorithms. The code that describes a piece of business logic is usually very simple. The structure of a COBOL-based billing system is much simpler than, for example, an embedded system for a Mars robot with complex real-time and multi-threading requirements. In enterprise applications, one usually finds very simple data structures, which again are different from other systems such as geographic information systems (GIS).

Let's consider an example in order to illustrate the difference between enterprise applications and other software: An enterprise application such as a Customer Relationship Management System (CRM), a billing system, a shipping system, or an insurance claims processing system. The stakeholders in these systems include different business units and potentially even the CEO, as well as different IT projects, IT maintenance, and operations. In these scenarios, we will be facing highly heterogeneous teams and often very political environments. The technology landscape will be highly heterogeneous as well, including many different application and middleware platforms. The business data and content will have a very long lifetime, especially when compared with the much shorter cycles of technology innovation. We need to deal with constantly changing functional requirements that are usually not well-defined. In addition, we will be facing many cross-dependencies between functional requirements, as well as heterogeneous technology platforms. The number of end users will be potentially very large, and the applications will have to be rolled out to large numbers of PCs, often more than 10,000.

Take, on the other hand, a desktop application, such as a word processor or spreadsheet application. A smaller, more homogeneous technical team will develop this application. It will be used by office workers as well, but the problem space is more well-defined. The application logic is self-contained, with very few cross-dependencies. Finally, there is no roll-out problem because the end user is typically responsible for the installation himself.

As we can see from these examples, enterprise software is unique in many respects, and therefore, it requires unique measures to ensure the efficiency of its development and maintenance.

1.3 The Importance of Enterprise Software Architectures

According to the second law of thermodynamics, *any closed system cannot increase its internal order by itself*. In fact, any activity that is geared toward ordering the system will increase its overall disorder (called *entropy*). In many respects, this law is also applicable to enterprise software, which often has very

similar characteristics. Consequently, *outside intervention* is continually required to help create a higher order and to ensure that development efforts are not lost.

In enterprise software, the architect takes on the role as an outside influencer and controller. It is his responsibility to oversee individual software projects from the strategic point of view of the overall organization, as well as from the tactical, goal-oriented viewpoint of the individual project. He has to balance different requirements while attempting to create an enduring order within the enterprise software landscape. The enterprise software architecture is the architect's most important tool at hand. Software architects are constantly confronted with changes to and expansion of functionality that increase system complexity and reduce efficiency. By refactoring current solutions, architects constantly strive to reduce complexity and thereby increase the agility of the system (see Figure 1-2).

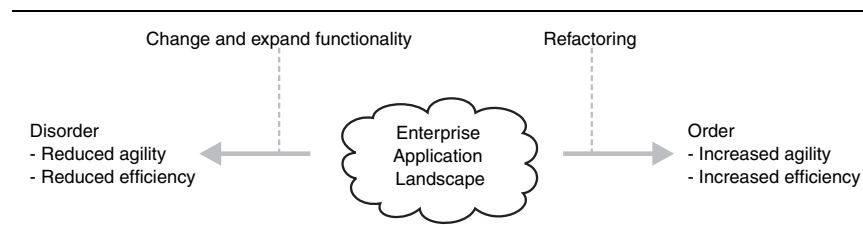


FIGURE 1-2 Software architects use refactoring to fight the constant increase in system complexity.

Apart from the events that increase complexity during normal usage of the architecture, single events can also have an important effect on enterprise IT. They might occur in major changes to existing jurisdiction, the end-of-life of a supported product, or the introduction of large chunks of computing infrastructure, such as in the course of a merger or acquisition. Such events require a major effort at very short notice to keep the architecture in a simple and maintainable state. Devastating consequences have been observed as a result of mergers and acquisitions: concise financial reporting being lost after a merger and raw system capacity being exhausted after an acquisition. Because it is unknown *a priori* when such effects will occur, it is vital to keep the enterprise architecture in a maintainable and changeable state all the time.

As we will see in the remainder of this book, Service-Oriented Architectures are particularly well suited to cope with the needs of such an ongoing incremental process of optimization.

1.4 The Requirements for an Enterprise Software Architecture

As a result of the aforementioned tight coupling with the internal organization, processes, and business model of the enterprise, an enterprise software architecture must fulfill very different requirements than, for example, a software architecture for a system that is controlled by a small number of highly qualified domain experts, such as the Mars robot or a video game engine.

In order to improve agility and efficiency, an enterprise software architecture must provide particular characteristics:

Simplicity. The enterprise architecture must be simple in order to allow efficient communication between key personnel. As previously discussed, many people are involved in the specification and construction of enterprise software. All these people have different roles and consequently different viewpoints with regard to the software. It is also likely that several different skill sets exist among personnel. These might range from IT coordinators of functional departments to technical architects. Many IT coordinators will have detailed business domain knowledge but no technical expertise. On the other hand, technical architects will probably have an excellent technical education but have little understanding of the vertical business. Nevertheless, all the people involved must be able to understand and manage the architecture at their respective levels (e.g., specifying new functionality at the business level and implementing and maintaining it).

Flexibility and maintainability. Every enterprise system is subject to ongoing change. It must continuously be adapted to new requirements due to the need of evolving markets, legal changes, or business reorganizations. Therefore, the architecture must lead to a highly flexible and maintainable system. The architecture must define distinct components that can be rearranged and reconfigured in a flexible manner. Local changes cannot be permitted to have an impact on the global system. Providing that the external API of a component remains stable, an internal change should not affect operations outside the component. In this context, one needs to understand that external interfaces of components must be designed very carefully. To a great extent, interfaces must be generic and not specific to a single usage scenario. However, defining generic interfaces requires excellent domain knowledge, experience, and to some extent, luck. Finally, the internal implementation of a component must allow efficient maintenance, making it is easy to add or modify functionality.

Reusability. Reusability has been a major objective of software engineering for decades, with varying degrees of success. It is in the interest of an enterprise to gain as much benefit from its software assets as possible. This can be achieved by creating an inventory of useful building blocks and continually reusing them. One obvious reason for reuse is reduced development and

maintenance cost, which can be accomplished by sharing common functionality in code libraries that are used across different projects. However, perhaps a more important aspect of reusability is the ability to share data across applications in real-time, thus reducing content redundancies. Having to maintain the same dataset in multiple databases becomes a nightmare in the long term. Unfortunately, it is not easy to achieve the goals of reuse. Large organizations have learned that reuse is not always efficient because it is particularly costly to administer, find, and understand the components that should be reused, and sometimes this cost outweighs the benefits.

Decoupling of functionality and technology. The architecture must make an enterprise organization independent of the technology. It must decouple the long lifecycle of the business application landscape from the shorter innovation cycles of the underlying technology. Moreover, an architecture that is designed to last longer than one or two of these technology innovation cycles must cope not only with changing technologies but also with the actual lifecycles of installed technologies, which can be much longer. It is therefore a major requirement that the architecture tolerate both heterogeneity and change to its technical infrastructure. Furthermore, the development of business functionality must be decoupled from the underlying technology. In particular, the architecture must avoid dependencies on specific products and vendors.

This book illustrates how a Service-Oriented Architecture can help achieve the design goals for enterprise software systems as described previously.

1.5 The Relation of Enterprise Architecture and Enterprise Standards

For many decades, enterprise IT organizations have attempted to improve agility and efficiency by homogenizing their systems through the introduction of enterprise-wide IT standards, but mostly with very limited success. Therefore, it is important to understand that an enterprise architecture is not equal to an enterprise standard, as we discuss in this section.

In the 1980s, with relational database systems becoming mainstream, we saw a wave of so-called *Enterprise Data Model* (EDM) projects. The idea of these standardization projects was to define one global data model for all the business entities in an enterprise, which was to be shared among all the different organizations and systems in a company. Almost all of these EDM projects failed, and today, there are usually as many different database schemas out there as there are databases in an enterprise. There are a variety of different reasons for the failure of these EDM projects, including political turf wars between different departments, conflicting interests between the different stakeholders ranging

from business representatives over application specialists to DBMS administrators, the sheer technical complexity of the undertaking, and the fact that due to the dynamics and complexity of modern enterprises, it is usually impossible to capture a snapshot of the complete state of the enterprise at a given point in time.

In the 1990s, we saw the next attempt to homogenize the enterprise application landscape, this time through enterprise-wide middleware standards. The concept of the *Enterprise Software Bus* became popular. The idea was that by agreeing on a ubiquitous, technology-independent, enterprise-wide standard for communication between software modules, the problem of application integration would be solved once and for all. However, the reality in almost all enterprises today is that in addition to application heterogeneity, we now face the problem of middleware heterogeneity as well. In many cases, middleware such as CORBA was only used to solve point-to-point integration problems on a per-project basis, instead of being established as a global software bus; as a result, many enterprises now have nearly as many incompatible middleware systems as they have applications.

In general, it seems fair to say that enterprise standardization efforts in IT have failed to deliver on their promise of homogenization and easy application integration. Too many generations of middleware—ranging from DCE over CORBA to SOAP and WSDL—have been touted as silver bullets but have failed to become established as *the* ubiquitous Enterprise Software Bus, leaving behind a high level of cynicism among the people involved.

As a reader of a book on Service-Oriented Architectures, you might now be asking yourself, “*So what is different this time?*” Is SOA not yet another enterprise-wide standardization effort, this time under the label of the *Enterprise Software Bus*? How are SOAP and WSDL—while maybe technically superior and more flexible—going to address the organizational challenges of global standards that made the Enterprise Data Model, the Enterprise Software Bus, and many other enterprise standardization efforts fail to a large extent?

This book takes the position that SOA is neither a technology nor a technology standard, but instead it represents a technology-independent, high-level concept that provides architectural blueprints, such as the ones outlined in the first part of this book. These architectural blueprints are focusing on the slicing, dicing, and composition of the enterprise application layer in a way that the components that are created and exposed as services in the SOA are not only technically independent but also have a direct relationship to business functionality. They enable the structuring of application components on the local level while also catering for global integration of these components. As we will show in this book, an SOA does not rely on the support of particular runtime protocols, such as SOAP or IIOP. Therefore, an SOA does not impose adherence to technical standards on the global level and is not based on strict norms and specifications (see Figure 1-3).

Applications that are structured according to the guiding SOA principles laid out in this book will be able to fit into any integration scenario, regardless of

the runtime protocols required. Having said this, work will still be required to bridge technology gaps, such as different communication protocols, but the efforts required to bridge these gaps will be marginal when compared to the complexity of integrating applications on the structural level.

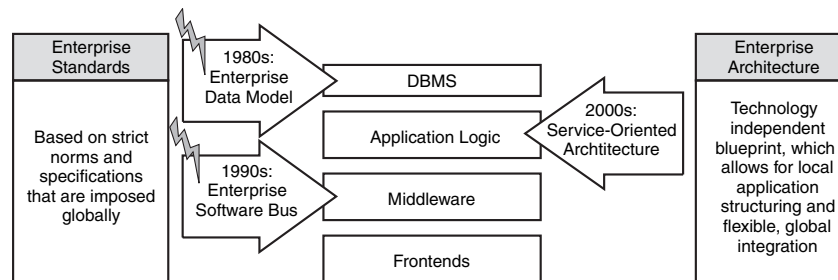


FIGURE 1-3 Enterprise Data Models and Software Buses were popular approaches to the challenges of enterprise computing in the 1980s and 1990s.

1.6 Organizational Aspects

When talking about enterprise IT, it is important to realize that many—if not most—of the problems associated with it are not of a technical nature but can be found on the organizational level instead. Quite naturally, we have already implicitly touched on many of these organizational aspects in our discussion so far (for example, when discussing the reasons for the failure of enterprise standards such as the *Enterprise Data Model* or the *Enterprise Software Bus*, which largely resulted from problems on the organizational and not the technical level).

The IT organization and the way projects are managed in a large enterprise are again very different from what one would find, for example, in a company that produced embedded systems or games. First and foremost, it is important to realize that most likely in no other part of the software industry will we find a development and maintenance process that is so closely aligned with the end customer. If an enterprise is developing a new financial reporting system, it will have to be done hand-in-hand with the finance department and any other stakeholders of the financial reporting system, possibly up to the CEO. A software team that is developing embedded control software for a dishwasher is unlikely to have daily meetings with a housewife about the exact functionality of the software.

An important consequence is that we are dealing with a much more complex and more ambiguously defined decision-making process, which is driven more often by business strategy and political agendas than by technical arguments. The organizational environment we are dealing with is extremely heterogeneous, and

many different opinions will have to be incorporated into any decision that is made, be it a decision about budgets, functional requirements, project priorities, or the interesting question of what actually defines the success of an IT project.

For all these reasons, it is vital that our *enterprise IT renovation roadmap* provides not only a *technical roadmap* but also an *organizational roadmap*, which outlines how the technical architecture is to be established on the enterprise level from the political and organizational point of view. The second part of this book provides an overview of this organizational roadmap.

1.7 Lifelong Learning

Enterprise software has always suffered from the mismatch between technical and business-related concepts and the different languages spoken by the people on both sides of the fence. As a result, we have not only faced inefficiencies, but we also have often lost important knowledge and consequently had to reinvent many solutions.

Many attempts have been made in the past to find a common denominator between business and technical concepts. For example, SQL was invented in the 1970s with the vision that it would give non-technical business analysts a tool to access, analyze, and manipulate business data directly. Today, SQL is largely seen as a tool for technical experts, and it has turned out that most of the entities found in relational databases are too fine-grained and closely intertwined with technical concepts to have a meaning on the business level.

It is a *key goal of an SOA to provide services that have a concrete meaning on the business level*. Because of this one-to-one mapping between business and technology entities, *SOA provides a unique chance for the first time in IT history to create artifacts that have an enduring value for both the business as well as the technology side*. SOA provides a chance to make things that have been learned the hard way usable for the organization in the long run.

Similarly to human beings, organizations will never be able to stop learning if they want to be successful for long. SOA provides an excellent platform for this lifelong learning on the organizational level because an SOA enables us to constantly compare the nominal and the actual and to react accordingly to fill the gaps or adapt the architecture to reflect changes in business strategy.

Consequently, the third part of this book provides a number of real-world case studies, which can provide a good starting point for learning the lessons resulting from other organizations' adoption of SOAs and the impact they had.

1.8 The Enterprise IT Renovation Roadmap

As we have outlined in this introduction, we need strong enterprise architecture concepts to address the structural problems of the enterprise IT landscape, accompanied by a strategy for how to establish the architecture on the organizational level. SOA provides these concepts, but we have to be aware that implementing an architecture like an SOA is an ongoing process, requiring constant guidance and overseeing. The SOA architect needs to bridge many conflicting requirements, resulting from frequent changes of business requirements, the evolution of application and infrastructure technology, and last but not least, changes to the architecture itself. We need ways to introduce step-wise improvements, which will bring us slowly but steadily closer to our goal. We will have to accept that the path we are about to enter will not be a straight path, and there will be many influences outside of our control. Nevertheless, the authors believe that the introduction of an SOA will bring many long-term benefits to an enterprise. Figure 1-4 depicts the overall vision for how our roadmap can bring an enterprise that is suffering from development inefficiencies back to a level of high efficiency and agility.

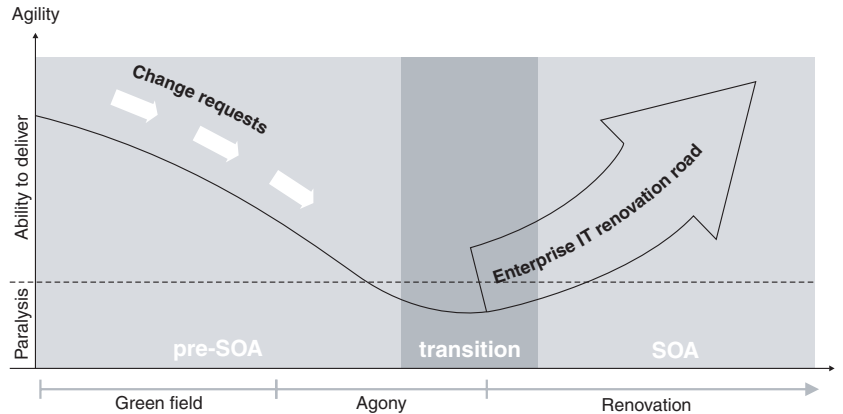


FIGURE 1-4 Service-Oriented Architecture is a key element of an enterprise IT renovation roadmap.

This book aims to flesh out an *enterprise IT renovation roadmap*. This roadmap is not only related to technology, but it also equally addresses organizational challenges. The roadmap anticipates constant changes in strategic directions and assumes that the underlying architecture will have to be constantly adapted to

include the results of lessons learned as we go along. Consequently, the three parts of this book are structured to capture these different requirements.

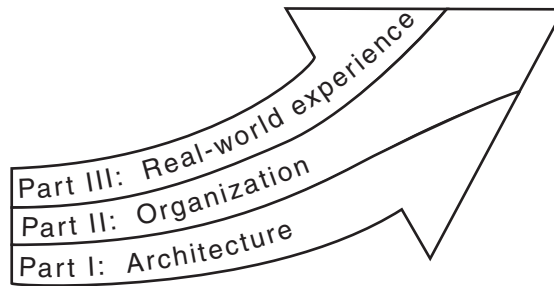


FIGURE 1-5 An Enterprise IT renovation roadmap needs to address three dimensions: architecture, organization, and real-world experience.

Part I of this book provides the *architectural roadmap*, mapping out the different *expansion stages* of an SOA, which will gradually help IT organizations to get back to a high level of agility and efficiency on the technical level.

Part II of this book looks at the *organizational roadmap*, providing an in-depth discussion of how the architecture can be established on the organizational level, the different stakeholders and how they must be involved, and how an SOA can be leveraged to drive project management more efficiently.

Finally, Part III of this book provides a number of case studies, which provide *real-world experiences* from large corporations that have already started their journey on the SOA-based enterprise renovation roadmap.